

CS 175: Project in AI: Project Proposal

10 Week Summer Session, 2026

Learning to Drive in Traffic

Measuring the Safety and Performance Tradeoff in Reinforcement Learning

Group Number: 7

Team member	UCI NetID	Student ID
Bala Kausik Vazrala	bvazrala	33516100
Christopher James Cho	cjcho2	48367996
Harrison Nguyen Xuan Hiep Le	harrisnl	10761825

1. Main Idea

An autonomous vehicle must constantly make tactical driving decisions. It must choose when to change lanes, when to accelerate, and when to yield. Every one of these choices balances two competing goals. The vehicle should make progress quickly, and it should never collide with surrounding traffic. These goals pull in opposite directions. A driver that prioritizes speed accepts risk. A driver that prioritizes safety may crawl behind slower traffic and never arrive on time.

Our project studies this tension directly. We will train autonomous driving agents in simulated highway traffic using reinforcement learning. The task for each agent is simple to state: drive as fast as possible without crashing. The interesting question is what happens when we change how much the training objective values speed relative to safety.

From an end user's point of view, the system is a driving policy. It observes the positions and speeds of nearby cars and selects a maneuver several times per second. The end product of the project is more than a single policy. We will train a family of agents whose objectives differ only in the penalty assigned to collisions. We will then measure how fast each agent drives and how often it crashes. The final deliverable is a tradeoff curve that shows what safety costs in speed and what speed costs in safety. This mirrors a real question in the autonomous vehicle industry. Manufacturers must decide how assertive a self-driving car should be, and that decision is ultimately encoded in an objective function.

2. Input and Output

At every decision step, the agent's input is a compact numeric description of nearby traffic. The simulator provides a table with one row per neighboring vehicle, up to a fixed count. Each row contains a presence flag, the vehicle's position, and its velocity, all expressed relative to the agent's own car. This representation is standard in the highway-env simulator and requires no image processing. At training time, the system also takes configuration inputs. The most important of these are the reward weights for speed and for collisions, the traffic density, and the number of lanes.

The policy's output is one of five discrete actions: change lane left, change lane right, accelerate, decelerate, or maintain course. A built-in low-level controller converts each chosen action into smooth steering and throttle commands.

The project as a whole will output trained model files for every configuration, learning curves logged during training, and a set of evaluation statistics for each agent. The central outputs are the average driving speed, the collision rate, and a plot of one against the other across all reward settings. We will also produce short rollout videos of trained agents. Potential applications include objective tuning for driving software, reward specification studies for safety-critical systems, and instructional material for reinforcement learning courses.

3. Basic Approach

We will use highway-env, an open source 2D driving simulator compatible with the Gymnasium interface [1, 3]. The simulator presents traffic from a top-down view. All vehicles except ours follow scripted behavior models. The Intelligent Driver Model governs their speed and the MOBIL model governs their lane changes. Our agent controls a single ego vehicle.

For learning, we will use the Deep Q-Network algorithm, known as DQN, through the Stable-Baselines3 library [2, 4]. DQN trains a neural network to estimate the expected future reward of each action in a given traffic situation. The agent then acts by choosing the action with the highest estimate, with occasional random actions for exploration. DQN is a natural fit because the action space is small and discrete. If time allows, we will add PPO as a second algorithm to check that our findings are not specific to DQN.

Our experimental design is deliberately narrow. We will copy known-good hyperparameters from published highway-env examples and freeze them. The only variable we change is the reward specification. We plan roughly five configurations that range from speed-heavy to safety-heavy, and we will train each configuration with three random seeds. Main experiments will run on the highway-fast-v0 variant of the environment so that a single training run finishes in under an hour on a laptop CPU.

We will clearly separate third party components from our own work. Gymnasium, highway-env, Stable-Baselines3, PyTorch, and TensorBoard are existing tools. Our original contributions are the experimental design, the training and evaluation code that ties these tools together, and the analysis of the results.

4. Evaluation Plan

Quantitative evaluation. Our primary metrics are the collision rate and the average speed. Collision rate is the fraction of evaluation episodes that end in a crash. Average speed is measured over the full episode. We will also record the mean episodic return, the mean episode length, and two behavior statistics: lane changes per episode and average following distance. Every trained agent will be evaluated on 200 held-out episodes with fixed evaluation seeds. We will report the mean and standard deviation of each metric across the three training seeds.

Baselines. We will compare against three baselines. The first is a random policy. The second is the simulator's scripted rule-based driver, which represents competent conventional behavior. The third is a DQN agent trained with the default reward weights, which anchors our sweep to published results.

Expected results. A random policy crashes within seconds in nearly every episode. We expect every trained agent to cut the random collision rate by more than 90 percent. We expect the default DQN agent to match or exceed the scripted driver's episodic return. Across the reward sweep, we expect the most safety-weighted agent to reduce the collision rate by at least half relative to the most speed-weighted agent, while giving up a measurable amount of average speed. We also expect collision rate to fall monotonically as the collision penalty grows. That monotonic trend is our key directional check. If it fails, the problem lies in the training pipeline rather than in the hypothesis.

Qualitative analysis. Our sanity cases are ordered. The training pipeline must first solve CartPole, a standard toy problem. Learning curves on highway-fast-v0 must then trend upward. Every trained agent must beat random. For visualization, we will show learning curves per configuration, the central speed versus collision rate scatter plot, and side-by-side rollout videos contrasting an aggressive agent with a cautious one. To inspect the internals of the algorithm, we will plot the network's action-value estimates in hand-picked situations. For example, the estimated value of accelerating should drop sharply when a car sits directly ahead.

Moonshot. Our moonshot is zero-shot transfer. We will take agents trained only on the highway scenario and evaluate them without retraining on the merge and roundabout scenarios. A clean result showing which reward settings generalize best would be an impressive headline. A textbook-quality Pareto frontier across all five configurations would be equally satisfying. Following the assignment guidelines, we treat these targets as estimates rather than promises.

5. Project Milestones

We plan the project as a six-week effort with a fixed compute budget. Every training run uses the same budget of 50,000 environment steps, decided in week one and never revisited. This rule exists to prevent open-ended tuning, which is the most common source of schedule slip in reinforcement learning projects. The final week is reserved as a buffer because overnight runs occasionally fail and must be repeated.

Week	Milestone	Key deliverables
1	Basic design	Shared repository created. Simulator and libraries installed on all three machines. DQN pipeline validated end-to-end on CartPole. Logging and checkpointing working. Step budget of 50,000 steps per run frozen.
2	Alpha	Random and scripted baselines measured. Default reward agent trained on highway-fast-v0. Evaluation script computes every metric automatically from a saved model.
3-4	Beta	Full matrix of five reward configurations and three seeds trained through queued overnight runs. First version of the tradeoff plot produced.
5	Analysis	Stress tests at doubled traffic density. Transfer experiments on merge and roundabout scenarios. Rollout videos rendered from saved policies.
6	Final	One showcase run on the full highway-v0 environment. Final report and demo preparation. Remaining time held as a buffer for reruns.

Within the team, one member will own the training infrastructure, one will own evaluation and plotting, and one will own analysis and writing. All members will review every part before submission.

References

- [1] E. Leurent. An Environment for Autonomous Driving Decision-Making. GitHub repository, 2018.
<https://github.com/eleurent/highway-env>
- [2] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, 22(268), 2021.
- [3] M. Towers, A. Kwiatkowski, J. Terry, et al. Gymnasium: A Standard Interface for Reinforcement Learning Environments. *arXiv:2407.17032*, 2024.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518, pages 529-533, 2015.